

Chapter 4

Software Testing Strategies

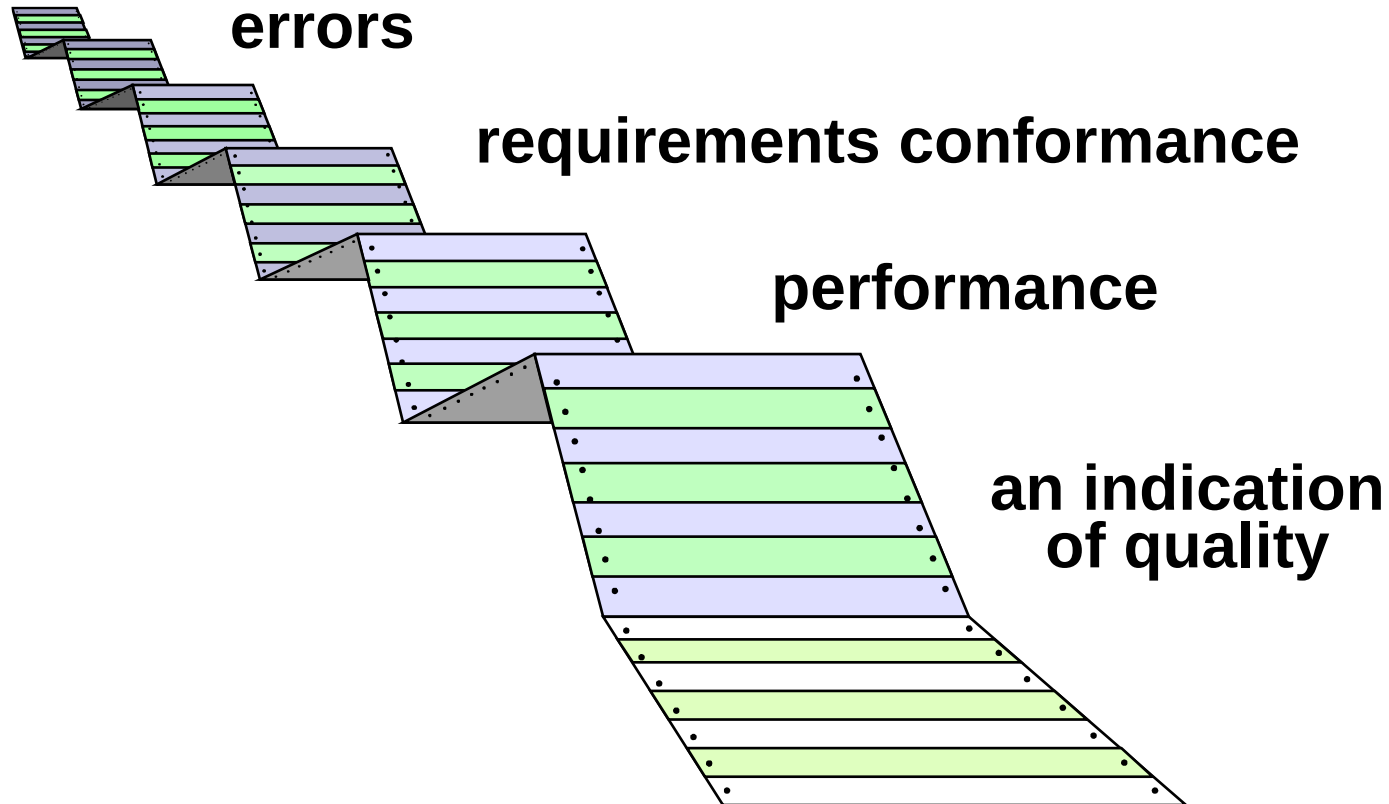
and Techniques

Software Engineering: A Practitioner's Approach, 6th edition
by Roger S. Pressman

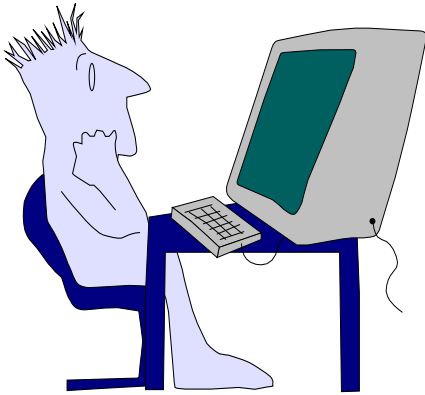
Software Testing

Testing is the process of exercising a program with the specific intent of finding errors prior to delivery to the end user.

What Testing Shows

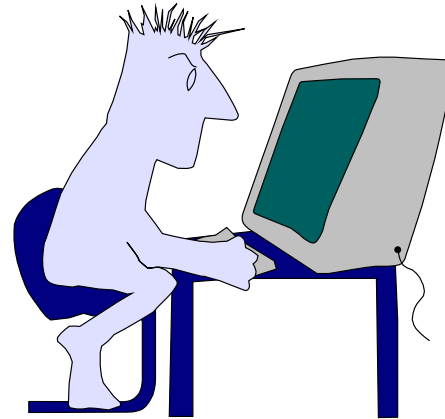


Who Tests the Software?



developer

**Understands the system
but, will test "gently"
and, is driven by "delivery"**



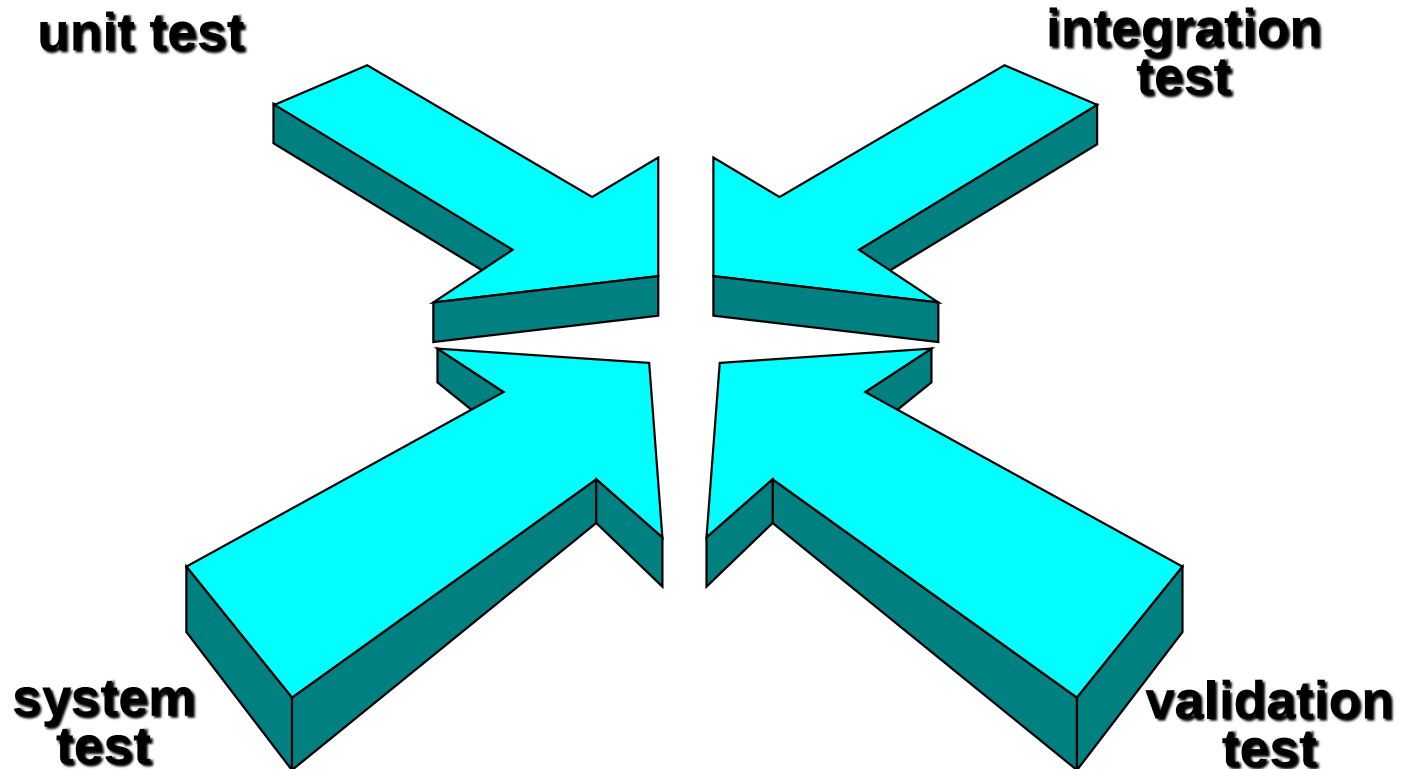
independent tester

**Must learn about the system,
but, will attempt to break it
and, is driven by quality**

Validation vs Verification

- **Verification** – Are we building the product right?
 - Is the code correct with respect to its specification?
- **Validation** – Are we building the right product?
 - Does the specification reflect what it should?

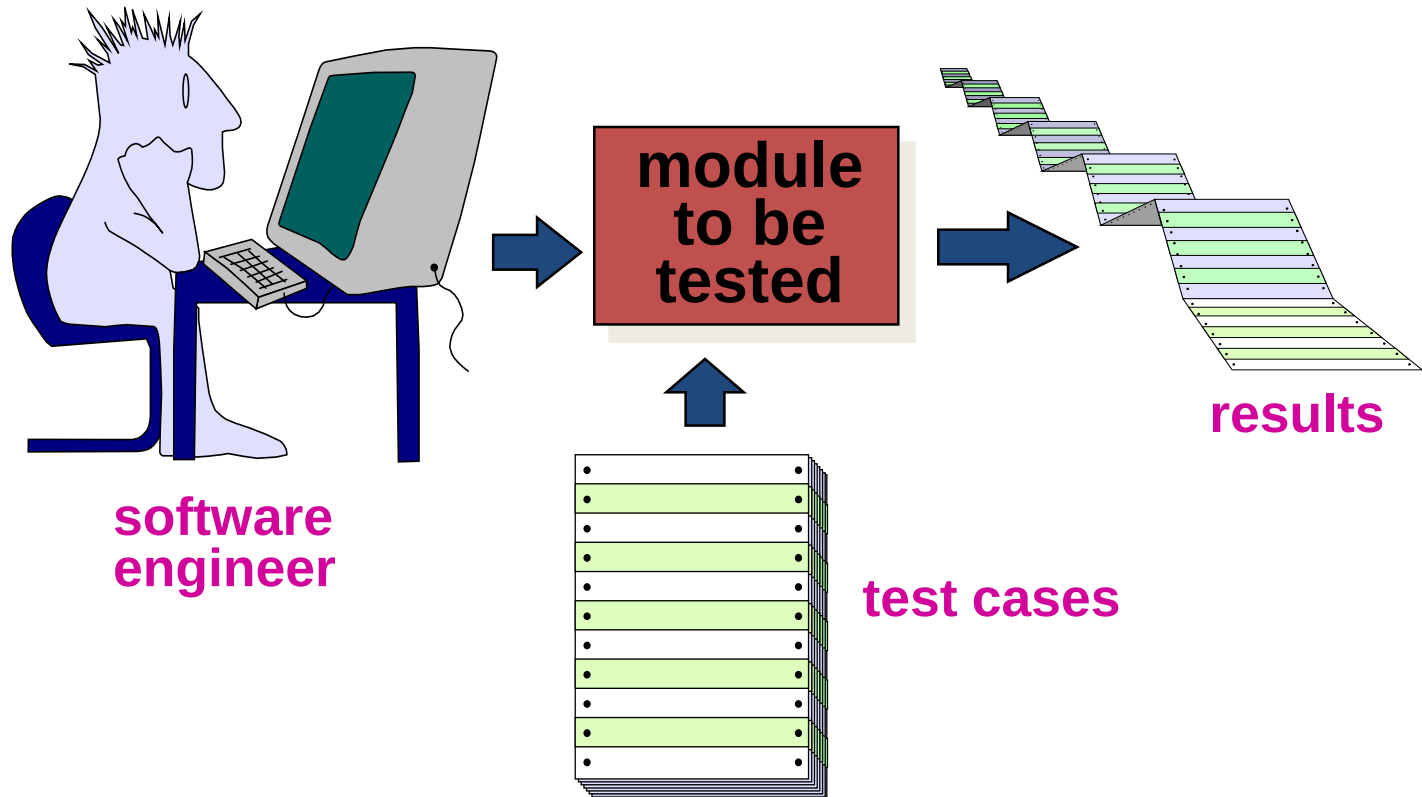
Testing Strategy



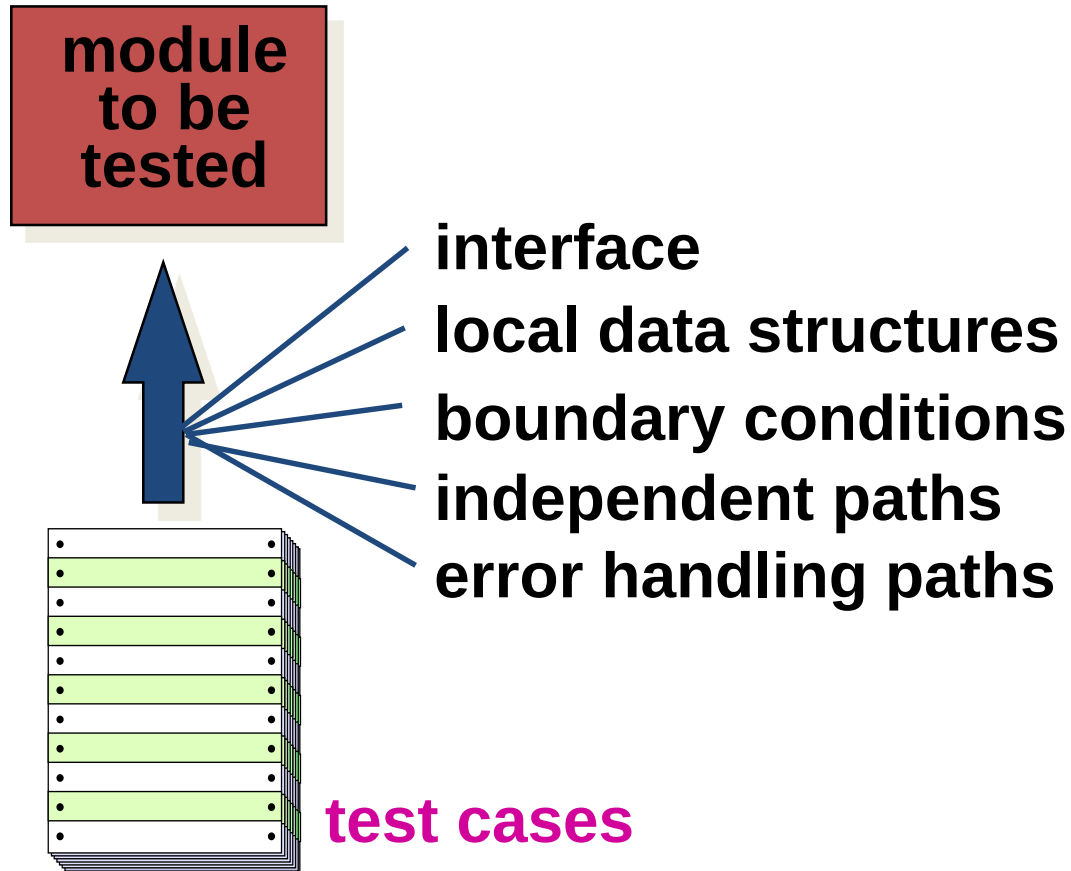
Testing Strategy

- Begin with unit testing and work your way up to system testing.
- 1) Unit testing – test individual components (modules in procedural languages; classes in OO languages).
 - 2) Integration testing – test collections of components that must work together.
 - 3) Validation testing – test the application as a whole against user requirements.
 - 4) System testing – test the application in the context of an entire system.

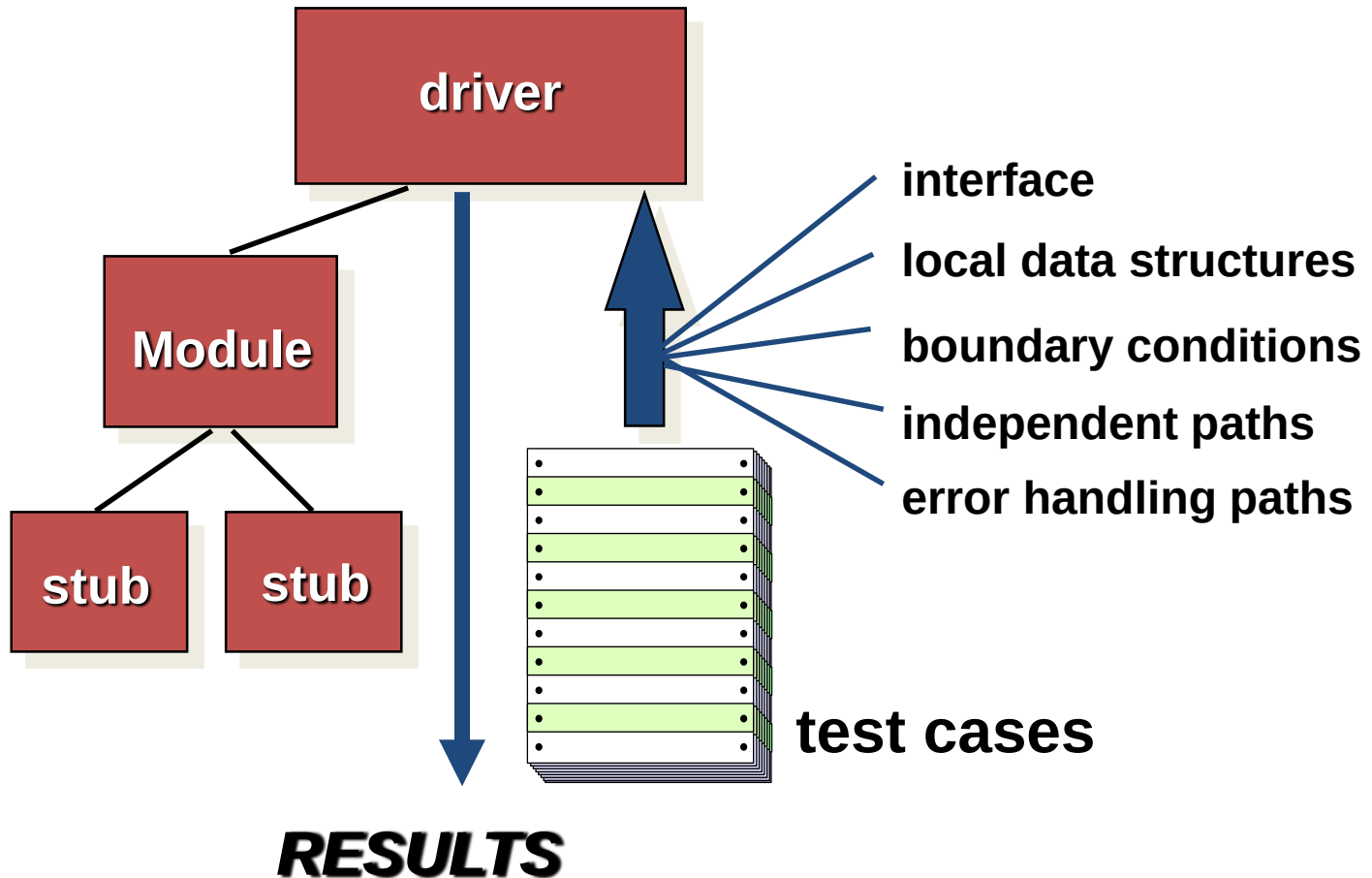
1) Unit Testing



Unit Testing



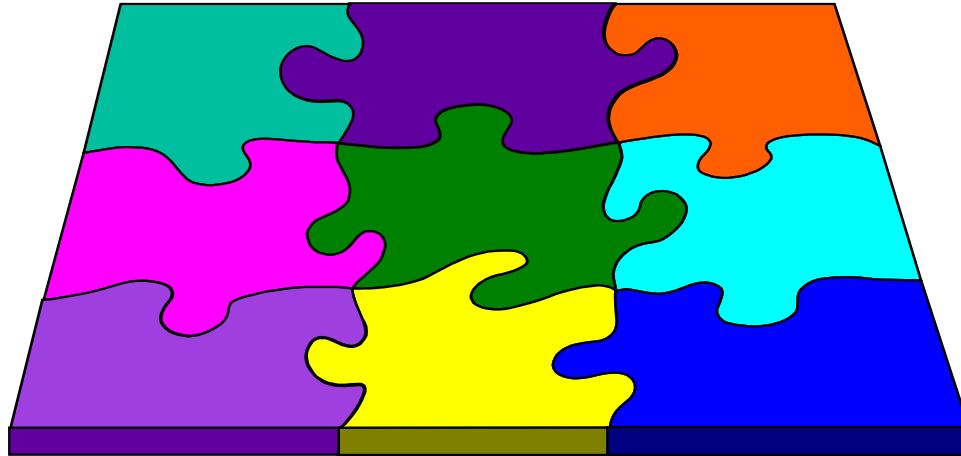
Unit Test Environment



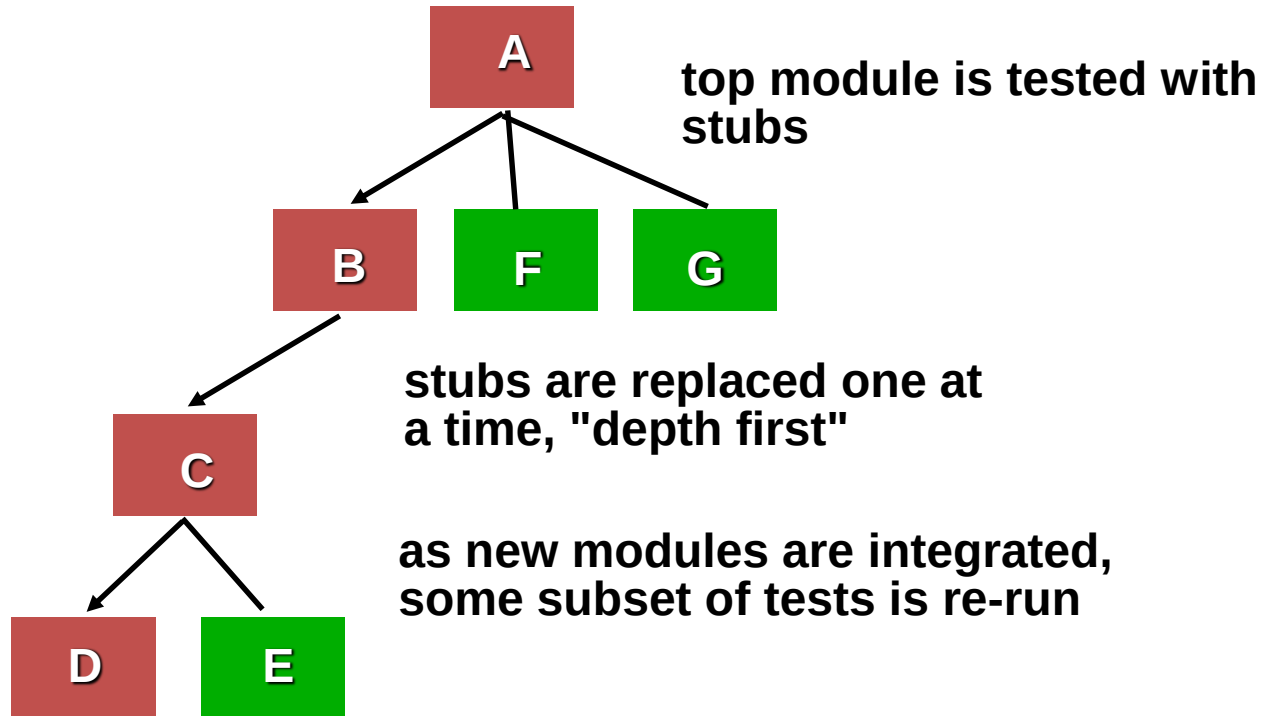
2) Integration Testing Strategies

Options:

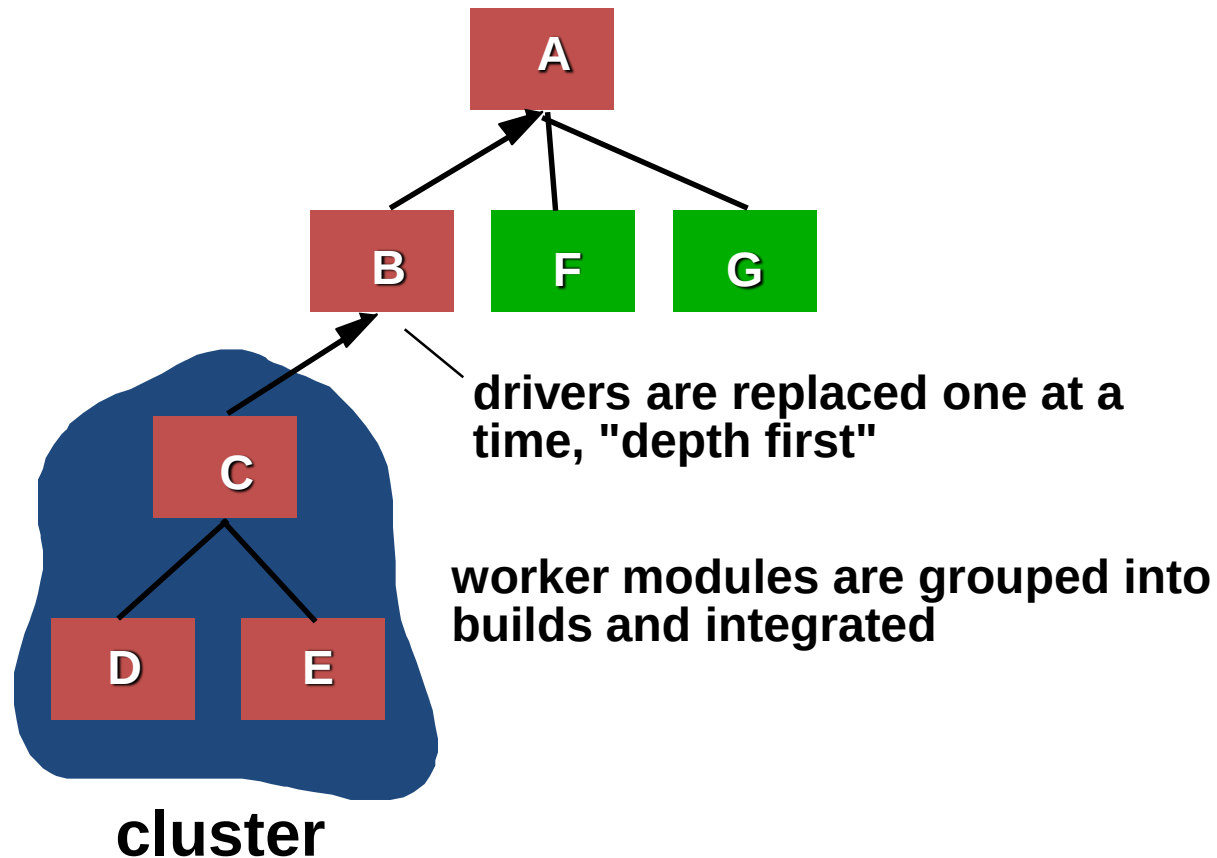
- the “big bang” approach
- an incremental construction strategy



Top Down Integration



Bottom-Up Integration



Regression Testing

- The selective retesting of a modified system to help ensure that no bugs have been introduced during modification.
 - Fixing one part of the code can break another.

High Order Testing

1) Validation testing

- Focus is on software requirements.

2) System testing

- Focus is on system integration.

3) Alpha/Beta testing

- Focus is on customer usage.

4) Recovery testing

forces the software to fail in a variety of ways and verifies that recovery is properly performed.

5) Security testing

verifies that protection mechanisms built into a system will, in fact, protect it from improper penetration.

6) Stress testing

executes a system in a manner that demands resources in abnormal quantity, frequency, or volume.

7) Performance Testing

test the run-time performance of software within the context of an integrated system.

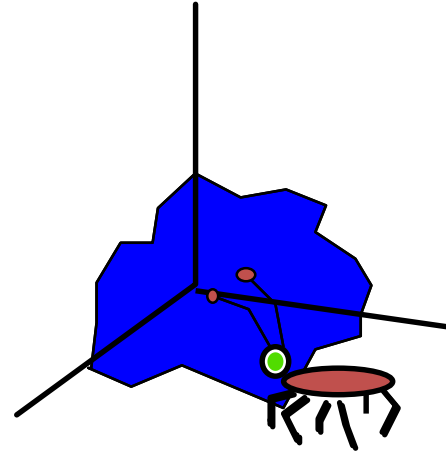
What is a “Good” Test?

- A good test is one that has a high probability of finding an error.

Test Case Design

"Bugs lurk in corners
and congregate at
boundaries ..."

Boris Beizer

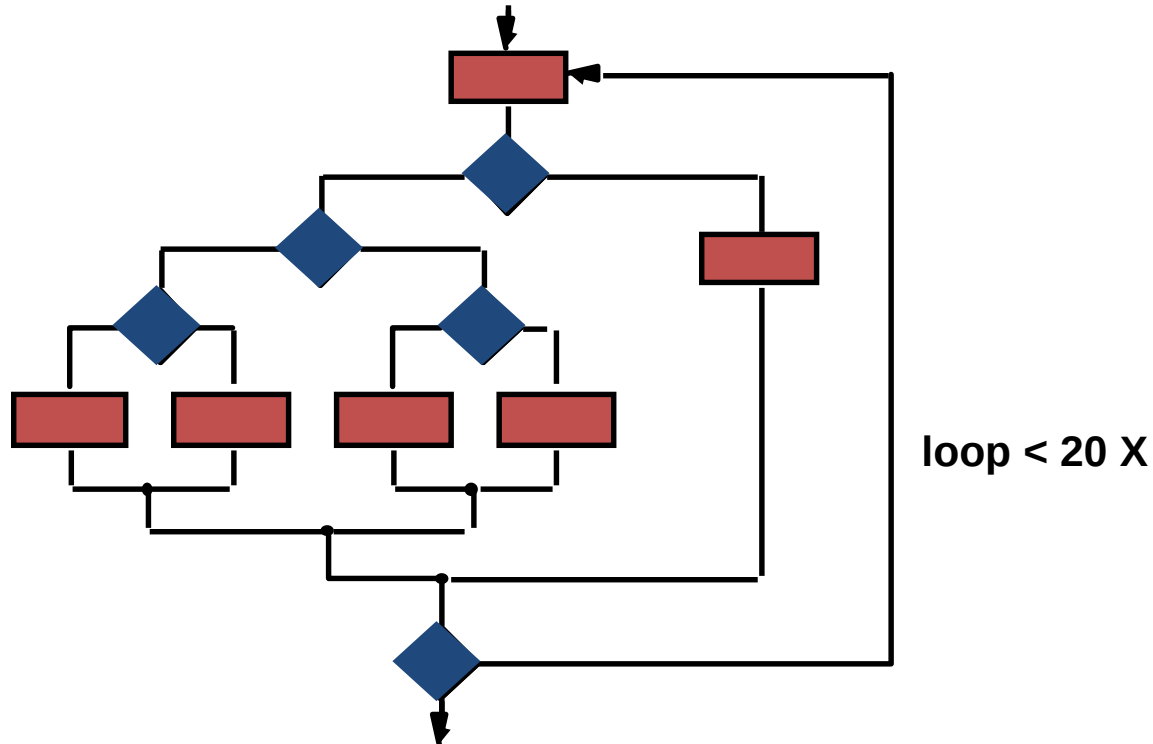


OBJECTIVE to uncover errors

CRITERIA in a complete manner

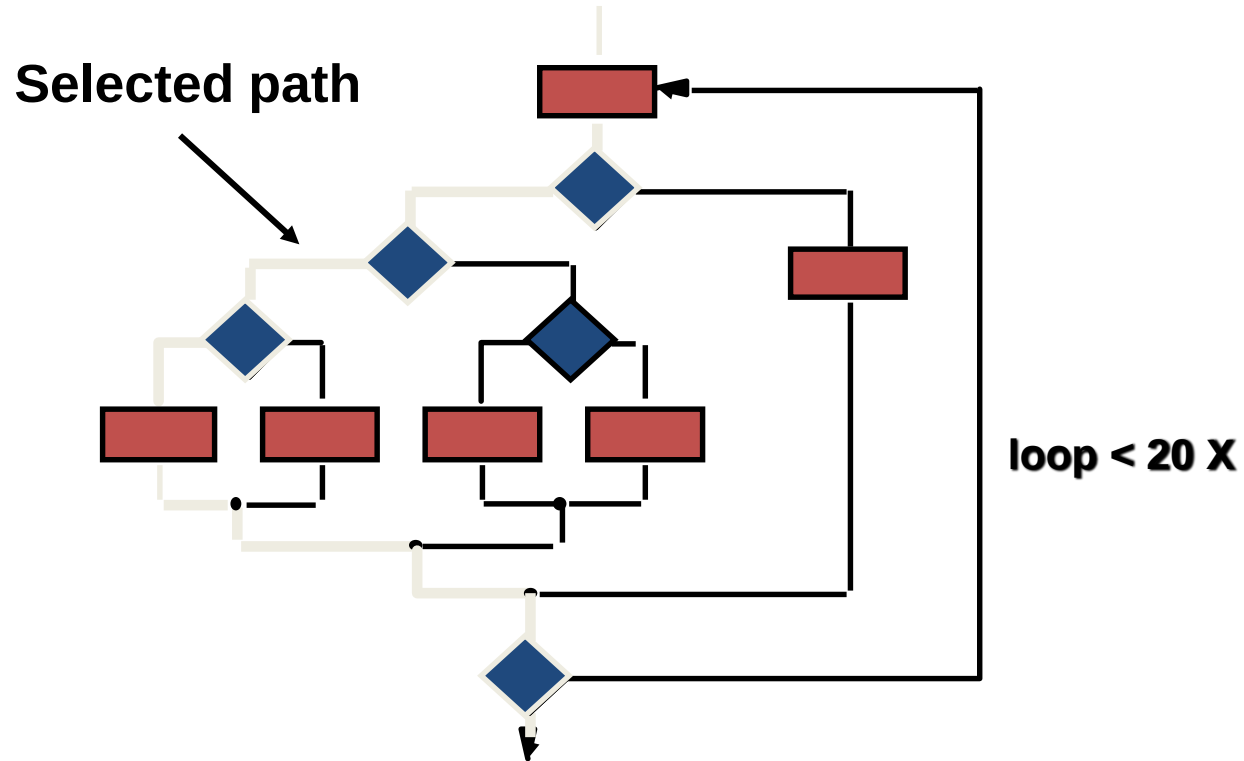
CONSTRAINT with a minimum of effort and time

Exhaustive Testing

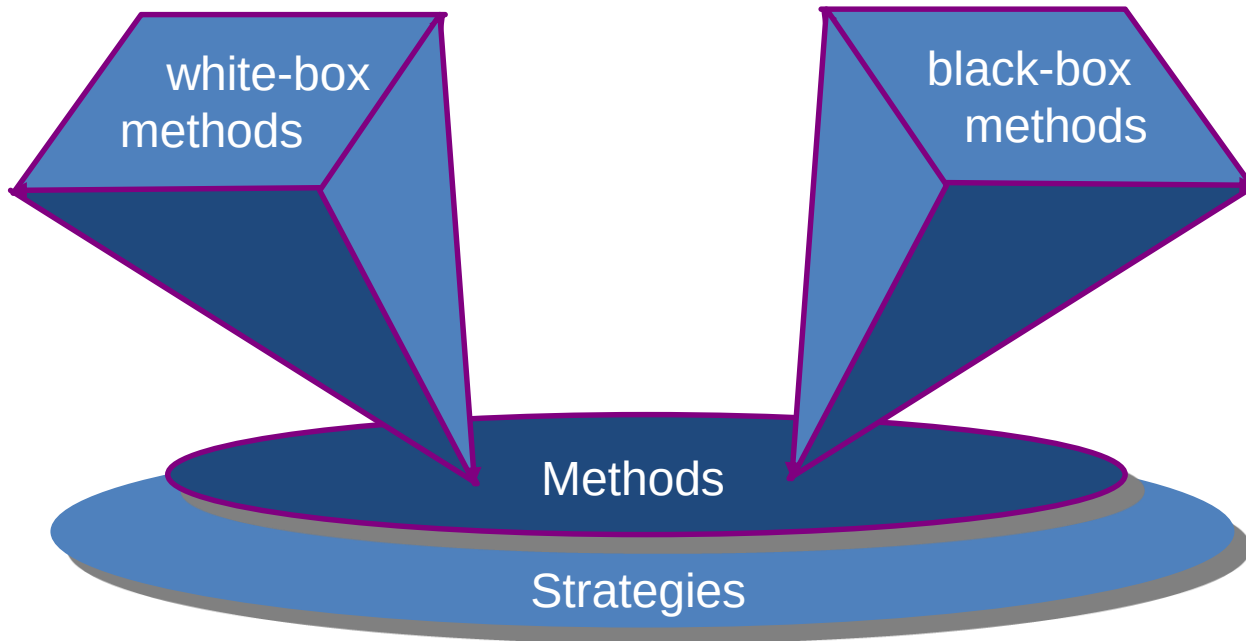


There are 10^{14} possible paths! If we execute one test per millisecond, it would take 3,170 years to test this program!!

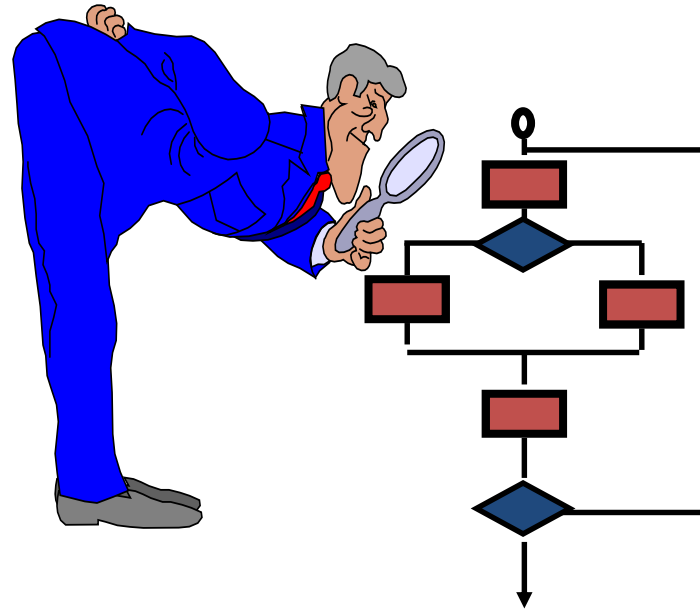
Selective Testing



Software Testing



White-Box Testing

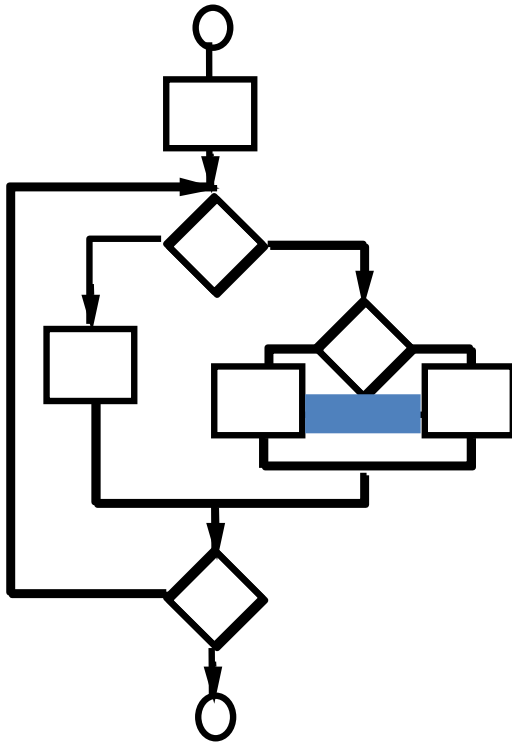


... our goal is to ensure that all statements and conditions have been executed at least once ...

Why Cover?

- ❑ logic errors and incorrect assumptions are inversely proportional to a path's execution probability
- ❑ we often believe that a path is not likely to be executed; in fact, reality is often counter intuitive
- ❑ typographical errors are random; it's likely that untested paths will contain some

Basis Path Testing



First, we compute the cyclomatic complexity:

number of simple decisions + 1

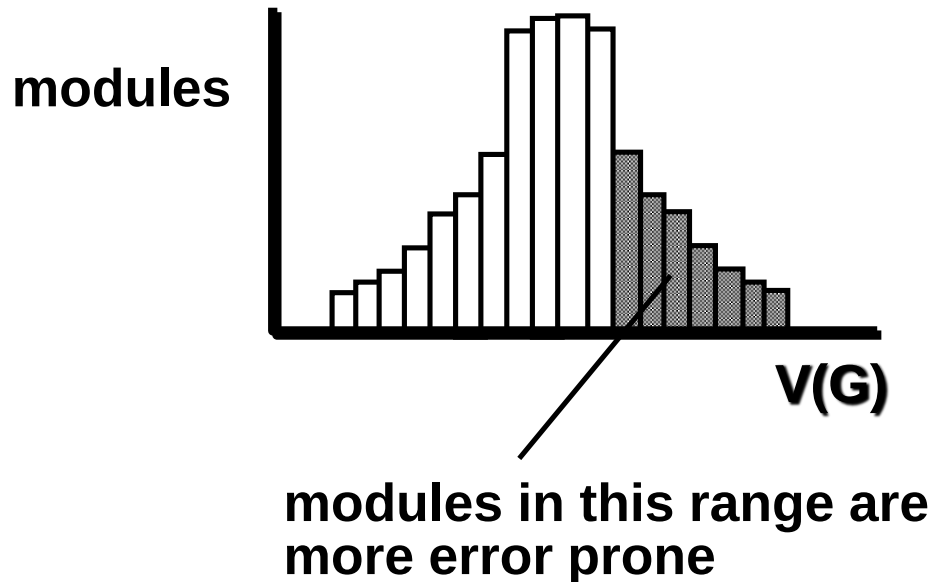
or

number of enclosed areas + 1

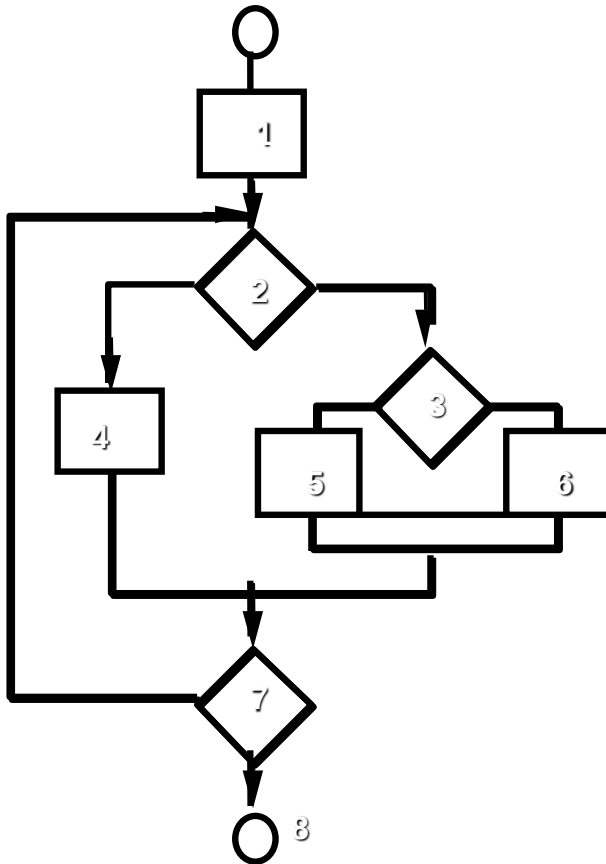
In this case, $V(G) = 4$

Cyclomatic Complexity

A number of industry studies have indicated that the higher $V(G)$, the higher the probability of errors.



Basis Path Testing



Next, we derive the independent paths:

Since $V(G) = 4$, there are four paths

Path 1: 1,2,3,6,7,8

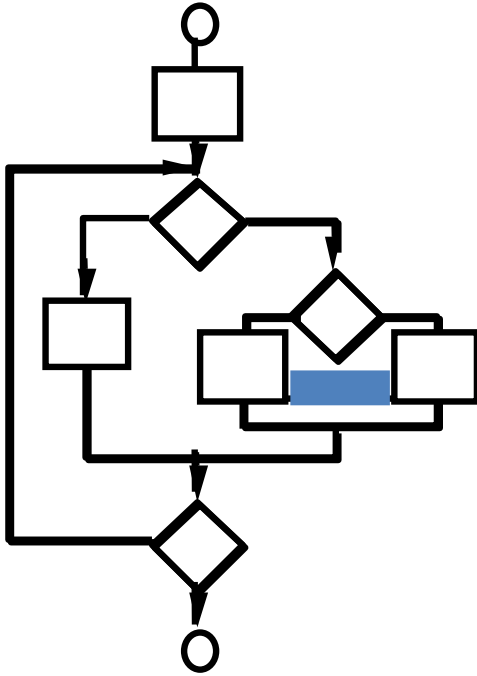
Path 2: 1,2,3,5,7,8

Path 3: 1,2,4,7,8

Path 4: 1,2,4,7,2,4,...7,8

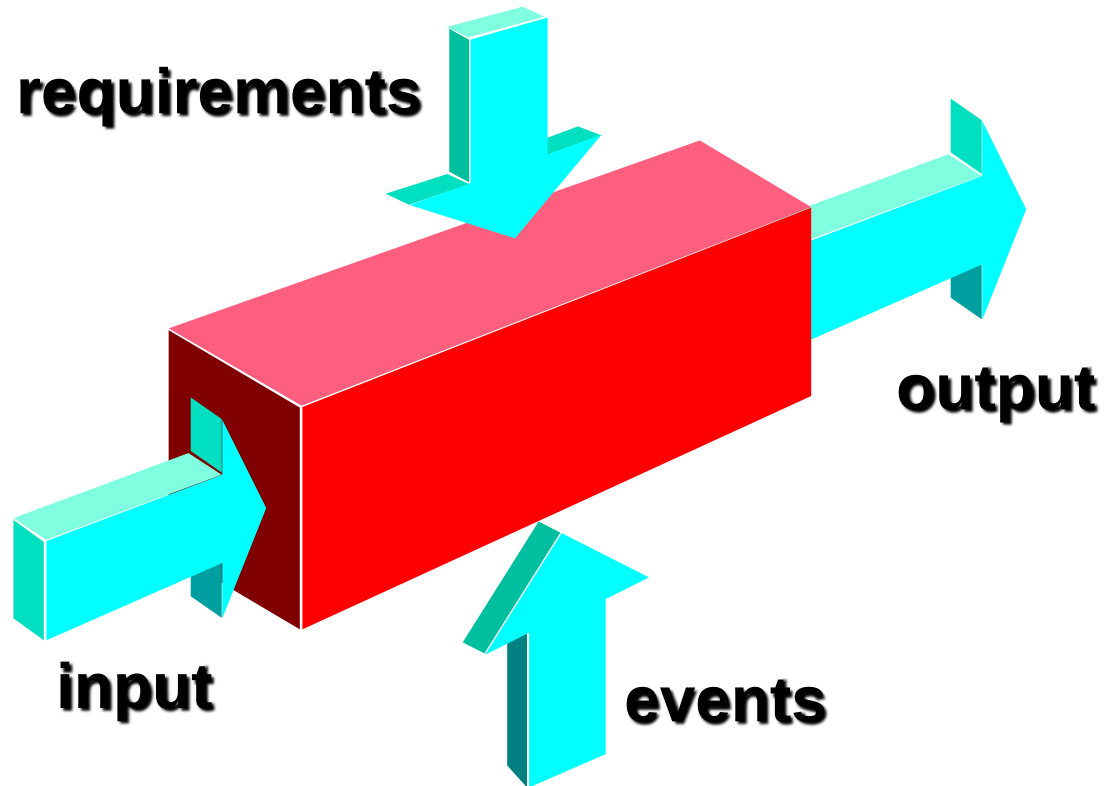
Finally, we derive test cases to exercise these paths.

Basis Path Testing Notes

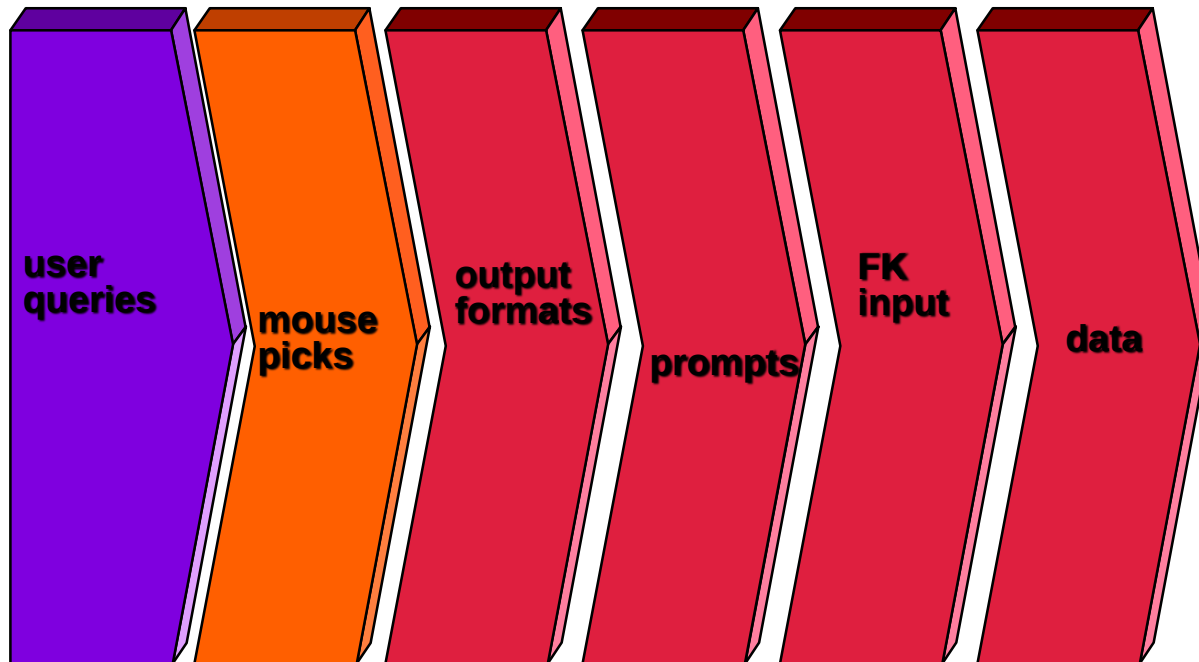


- ❑ you don't need a flow chart, but the picture will help when you trace program paths
- ❑ count each simple logical test, compound tests count as 2 or more
- ❑ basis path testing should be applied to critical modules

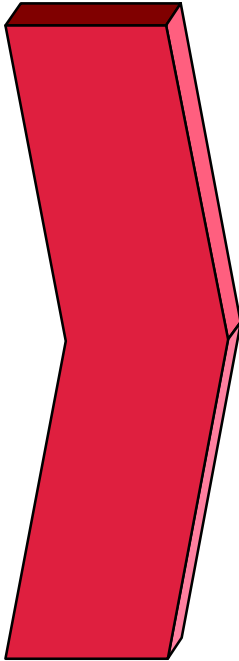
Black-Box Testing



Equivalence Partitioning



Sample Equivalence Classes



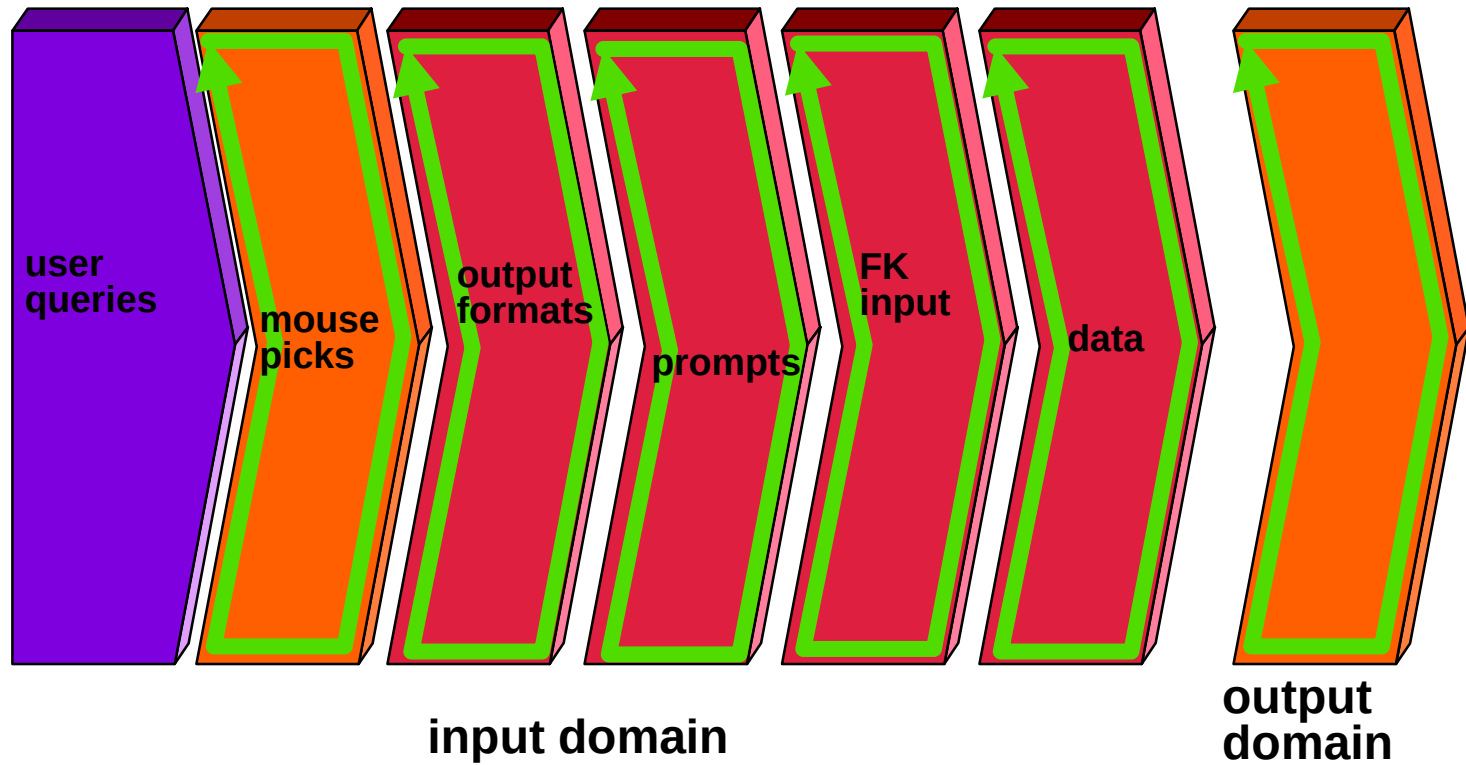
Valid data

user supplied commands
responses to system prompts
file names
computational data
 physical parameters
 bounding values
 initiation values
output data formatting
responses to error messages
graphical data (e.g., mouse picks)

Invalid data

data outside bounds of the program
physically impossible data
proper value supplied in wrong place

Boundary Value Analysis



OOT Methods: Behavior Testing

The tests to be designed should achieve all state coverage [KIR94]. That is, the operation sequences should cause the Account class to make transition through all allowable states

